

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_98c1f280-b9f\agent-aae1088a3de0e2161.jsonl`  
- SHA-256: `0e78ed5c9389ce8f74282b0514534809a27d3e8968bf521fb1415f34a001d7b0`  
- Source modified: `2026-06-14T06:33:50+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_98c1f280-b9f`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-14T06:32:15.326Z)

In the current badminton-highlight-indexer repo (cwd), audit ONE signal/component. Verify and return:

- present: the code exists.
- default_off: it does NOT run in the served/default path by default (e.g. cue_flags default False, default_segmenter is NOT 'fusion', the feature is not weighted by the shipped scorer unless explicitly enabled).
- no_special_casing: the signal enters the DECISION as a confidence-scored / feature COLUMN the learned scorer weights ? NOT a hard if/else branch hard-coded into the decision path.
- retained: NOTHING deletes or disables the code because the n=6 measured ?F1 was non-significant; the producer + feature math still exist and are re-measurable when the corpus grows.

Give evidence as concrete file:line refs with short quotes. List concerns. Be precise and skeptical; read the actual files.

COMPONENT: the FusionSegmenter + config defaults. Read backend/pipeline/segmenters/fusion_hybrid.py (family, cue_flags, Gate A/B), backend/pipeline/segmenters/trajectory_hybrid.py (identity hooks), and backend/config/models.py (FusionConfig cue_flags defaults + IndexingConfig default_segmenter). Confirm the fusion path ships default-OFF (default_segmenter unchanged; cue_flags default False).

2. assistant (2026-06-14T06:32:17.641Z)

I'll audit the FusionSegmenter + config defaults component. Let me start by reading the key files.

3. user (2026-06-14T06:32:18.612Z)

```
1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).  
2  
3      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,  
4      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`  
5      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each  
6      shuttle-seeded window via two overridable seams the engine exposes:  
7  
8      - ``enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,  
9      GPU-free, computed from the trajectory we already have); and, only when the person cue  
10     is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features  
11     (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned  
12     fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads  
the  
13     torchvision/supervision model ? so the default (person-OFF) path never loads a  
detector  
14     model nor touches the GPU. (torch/cv2 themselves are imported by the registry  
regardless,  
15     via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
```

```

16     - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17     (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18     dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19     **Persisted candidates remain the full superset** regardless, so the offline
experiment
20     harness can re-score any variant.
21
22     Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23     nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26     experiment harness (EXPERIMENT_HARNESS.md), never silently.
27     """
28     from typing import Any, Dict, List, Optional, Tuple
29
30     from backend.pipeline.segmenters.base import segmenter_registry
31     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32     from backend.pipeline.detectors.base import DetectorRunner
33     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35     from backend.pipeline.detectors import rally_gate
36     from backend.pipeline.detectors import fusion_features as ff
37
38
39     class FusionSegmenter(TrajectoryHybridSegmenter):
40         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42         family = "fusion"
43         display_label = "Fusion"
44
45         def __init__(self, db: Any, config: Any):
46             super().__init__(db, config)
47             # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48             self._person: Optional[Any] = None
49             # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50             # over the whole trajectory once per candidate window (it depends only on
`points`).
51             self._axis_cache_key: Optional[int] = None
52             self._axis_cache: Optional[tuple] = None
53
54         @property
55         def name(self) -> str:
56             return "fusion_hybrid"
57
58         @property
59         def description(self) -> str:
60             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                     "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                     "AI provider sets semantic boundaries.")
63
64         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65             substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66             if substrate == "tracknet":

```

```

67         cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68         return TrackNetRunner(cfg), {
69             "weights_path": cfg.weights_path,
70             "queue_length": cfg.queue_length,
71             "fusion_substrate": "tracknet",
72         }
73     wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74     return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                               frame_width: float, video_path: str,
78                               idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82         # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83         gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
84                                             min_crossings=0,
estimate=self._axis_estimate(points))
85         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87         fcfg = idx_cfg.get("fusion", {})
88         if fcfg.get("cue_flags", {}).get("person", False):
89             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90         return stats
91
92     def _axis_estimate(self, points: List[Any]) -> tuple:
93         """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
94         computed once per video, not once per candidate window (it depends only on
points)."""
95         key = id(points)
96         if self._axis_cache_key != key or self._axis_cache is None:
97             self._axis_cache_key = key
98             self._axis_cache = rally_gate.estimate_play_axis(points)
99         est = self._axis_cache
100         assert est is not None
101         return est
102
103     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
104                        frame_width: float, video_path: str,
105                        fcfg: Dict[str, Any]) -> Dict[str, float]:
106         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
107         frame range and compute the scale/translation-invariant person features. Lazily
builds
108         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
109         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
110         from backend.pipeline.detectors.person_detector import PersonDetector
111         if self._person is None:
112             self._person = PersonDetector(self.config)
113
114         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
115         # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
116         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
117         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)

```

```

118         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
119         fw = det["frame_width"] or frame_width
120         fh = det["frame_height"]
121         # If we can't trust the frame dims, every normalised feature would silently
collapse
122         # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
123         if fw <= 0 or fh <= 0:
124             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
125
126         poly = fcfg.get("court_polygon")
127         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
128         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
129         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
130         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
131                                         court_polygon=polygon, net=net)
132
133     def _select_for_handoff(self, windows: List[Dict[str, Any]],
134                           window_stats: List[Dict[str, Any]],
135                           idx_cfg: Dict[str, Any]) -> List[int]:
136         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
137         (0) keep every window ? identical to the substrate. Persisted candidates are
138         untouched (full superset)."""
139         fcfg = idx_cfg.get("fusion", {})
140         min_crossings = int(fcfg.get("min_crossings", 0))
141         min_players = int(fcfg.get("min_players", 0))
142         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
143
144         keep: List[int] = []
145         for i, st in enumerate(window_stats):
146             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
147                 continue
148             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
149                 continue
150             keep.append(i)
151         return keep
152
153
154     segmenter_registry.register(FusionSegmenter)
155

```

4. user (2026-06-14T06:32:18.647Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])

```

```

21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"
42         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43         velocity_threshold: float = 0.02
44         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45         # decoded frame cache) are deleted automatically ? they are pure intermediates,
46         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48         keep_frames: bool = False
49         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50         # 0 = disabled.
51         min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66         sport: str = "badminton"
67         wsl_stage_dir: str = "~/clips"
68         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69         velocity_threshold: float = 0.02
70         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
71         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
72         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
73         keep_frames: bool = False
74         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
75         min_free_gb: float = 0.0
76
77
78     class FusionConfig(BaseModel):
79         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).

```

```

80
81 Every default reproduces control behaviour: the fusion segmenter persists the
82 shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
83 true ? the person-cue features, but it does NOT gate the served handoff unless a
84 threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
85 is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
86 supplied ? off-court persons (spectators / adjacent courts) are excluded.
87 ""
88 # Which trajectory substrate seeds the windows (shuttle stays primary).
89 substrate: Literal["wasb", "tracknet"] = "wasb"
90 # Windowing velocity threshold for the fusion family (the engine reads
91 # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
92 # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
93 velocity_threshold: float = 0.02
94 # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
95 # enabled ? so the default path never imports torch / touches the GPU.
96 cue_flags: dict[str, bool] = Field(default_factory=dict)
97 # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
98 # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
99 min_crossings: int = Field(default=0, ge=0)
100 # Served Gate B: when the person cue is on, drop windows with median in-court
players
101 # < this. 0 = OFF.
102 min_players: int = Field(default=0, ge=0)
103 # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
104 court_polygon: Optional[list[list[float]]] = None
105 # Net line for the person two-sided-motion split (person features only ? the shuttle
106 # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
107 net_axis: Literal["x", "y"] = "y"
108 net_position: float = Field(default=0.5, ge=0.0, le=1.0)
109
110
111 class IndexingConfig(BaseModel):
112     default_segmenter: str = "yolo_hybrid"
113     use_gpu: bool = True
114     motion_threshold: float = 0.08
115     min_segment_duration: float = 3.0
116     dead_time_merge_gap: float = 2.0
117     chunk_duration: float = 90.0
118     chunk_overlap: float = 10.0
119     detector_model: str = "fasterrcnn_resnet50_fpn"
120     detector_score_threshold: float = 0.5
121     yolo_velocity_threshold: float = 0.15
122     yolo_frame_skip: int = 3
123     yolo_tracker: str = "bytetrack.yaml"
124     yolo_prefetch_workers: int = 2
125     min_action_window_duration: float = 2.0
126     log_yolo_candidates: bool = True
127     store_yolo_candidates: bool = True
128     ai_handoff_padding: float = 2.0
129     ai_max_workers: int = 5
130     # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
131     # chunks of a high-bitrate (4K) source blow past the provider upload cap
132     # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
133     # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
134     # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
135     # Matches gemini_refine's --clip-scale-width default.
136     ai_chunk_scale_width: int = 1280
137     # Optional debug knob: trim every video to the first N seconds before processing.
138     debug_duration: Optional[float] = None

```

```

139
140     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
141     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
142     fusion: FusionConfig = Field(default_factory=FusionConfig)
143
144     @field_validator("default_segmenter")
145     @classmethod
146     def segmenter_must_be_registered(cls, v: str) -> str:
147         # Registry check happens at runtime in main/segmenter selection.
148         # Here we just allow any string for forward compatibility.
149         return v
150
151     @model_validator(mode="after")
152     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
153         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
154         # step makes `while t < duration: t += step` loop forever, growing memory before
any
155         # GPU work. Reject the bad config at load time instead.
156         if self.chunk_overlap >= self.chunk_duration:
157             raise ValueError(
158                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
159                 f"({self.chunk_duration}); otherwise chunking never advances."
160             )
161         return self
162
163
164     class OutputConfig(BaseModel):
165         default_highlights_name: str = "highlights.mp4"
166         db_name: str = "sports_indexer.db"
167         # Directory where the DB, highlight reels, and processed clips are written.
168         # The CLI's --output-dir overrides this at startup (see backend/main.py:main).
169         output_dir: str = "output"
170
171
172     class WatermarkConfig(BaseModel):
173         """Branding overlay burned into each highlight clip during the per-clip re-encode
174         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
175         (under `stitching.watermark` in config.json) to turn it off. The text is fully
176         configurable. The concat stage stays stream-copy (-c copy)."""
177         enabled: bool = True
178         text: str = "Generated by Khelsutra.guru"
179         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
180         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
below
181         font: str = "Sans" # fontconfig family used when font_path is empty
182         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of
frame height
183         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
184         box: bool = True # faint backing box behind the text for legibility
185         margin: int = Field(default=24, ge=0) # px inset from the chosen corner
186         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
187
188
189     class StitchingConfig(BaseModel):
190         begin_padding: float = 0.5
191         end_padding: float = 0.5
192         use_cache: bool = False
193         min_shot_count: int = 1
194         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
195
196
197     class LoggingConfig(BaseModel):

```

```

198     """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
199     # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
200     # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
201     # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
202     # them to WARNING; set true to restore full chatter for request-flow debugging.
203     verbose_http: bool = False
204
205
206     class SecurityConfig(BaseModel):
207         # When set, /api/process video_path must resolve under this directory (hosted/tenant
208         # mode). Default None preserves the single-user local 'any local file' workflow.
209         allowed_video_root: Optional[str] = None
210
211         # --- Chunked upload (B2, PR-2) ---
212         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
213         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
214         # contain upload_dir or /api/process will reject the uploaded paths.
215         upload_dir: str = "output/uploads"
216         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
217         # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
218         max_upload_mb: int = 4096
219         max_part_mb: int = 95
220         allowed_upload_extensions: List[str] = ["mp4", "mov"]
221
222
223     class AppConfig(BaseModel):
224         """Root configuration object.
225
226         All runtime configuration should be accessed via an instance of this class
227         (or the loader) rather than raw dict lookups.
228         """
229
230         sport: Sport = "badminton"
231         ai_provider: str = "gemini"
232         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
233         # the alias never deprecates underneath us. Eval reports record the run date
234         # alongside.
235         ai_model: str = "gemini-flash-latest"
236
237         validation: ValidationConfig = Field(default_factory=ValidationConfig)
238         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
239         output: OutputConfig = Field(default_factory=OutputConfig)
240         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
241         security: SecurityConfig = Field(default_factory=SecurityConfig)
242         logging: LoggingConfig = Field(default_factory=LoggingConfig)
243
244         # Allow extra keys during transition so existing config.json files do not break.
245         model_config = {
246             "extra": "allow",
247             "validate_assignment": True,
248         }
249
250         # Temporary migration shim for legacy dict-style access (config.get("section",
251         # {}).get(...)).
252         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
253         # attribute access / the typed config. A nested *section* is returned as a plain
254         # dict
255         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
256         # section model instead breaks it (a model has no .get), which bit every validator
257         # on
258         # real runs. Scalar leaves are returned as-is.
259         def get(self, key: str, default: Any = None) -> Any:
260             _MISSING = object()
261             val: Any = _MISSING

```



```

258         if key in type(self).model_fields or hasattr(self, key):
259             val = getattr(self, key, _MISSING)
260         if val is _MISSING:
261             # Legacy convenience: find a leaf key inside any nested section. Sections
are
262             # discovered dynamically from the model fields, so a future top-level
section
263             # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
264             for fname in type(self).model_fields:
265                 section = getattr(self, fname, None)
266                 if isinstance(section, BaseModel) and hasattr(section, key):
267                     val = getattr(section, key)
268                     break
269         if val is _MISSING:
270             return default
271         if isinstance(val, BaseModel):
272             return val.model_dump(mode="json")
273         return val
274
275     def __getitem__(self, key: str) -> Any:
276         val = self.get(key)
277         if val is None and key not in type(self).model_fields:
278             # Re-raise like a real dict for unknown top-level
279             raise KeyError(key)
280         return val
281
282
283     # Convenience type alias for callers that want to annotate
284     Config = AppConfig
285

```

5. assistant (2026-06-14T06:32:21.799Z)

Now let me look at the parent class `trajectory\_hybrid.py` for the identity hooks and base segmenter behavior, plus check where `default\_segmenter` is consumed and how the scorer treats these features.

6. user (2026-06-14T06:32:22.210Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os
21    import time
22    import logging
23    import concurrent.futures
24    from typing import Any, Dict, List, Optional, Tuple
25

```

```

26     import cv2
27
28     from backend.pipeline.segmenters.base import BasePlaySegmenter
29     from backend.utils.video import get_video_duration, extract_video_chunk
30     from backend.sports import sport_registry
31     from backend.providers import provider_registry
32     from backend.pipeline.detectors.base import DetectorRunner
33
34     logger = logging.getLogger(__name__)
35
36
37     def _fmt_ts(seconds: float) -> str:
38         seconds = max(0, int(seconds))
39         h, rem = divmod(seconds, 3600)
40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46         boundaries."""
47
48         #: config section under `indexing` (velocity_threshold lives there), the
49         #: output subdir for trajectories, and the metadata detector-tag prefix.
50         family: str = ""
51         #: human-readable label used in log lines.
52         display_label: str = ""
53
54         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55         Any]]:
56             """Return (runner, detector-specific detection_params extras)."""
57             raise NotImplementedError
58
59         @staticmethod
60         def _probe_fps_width(video_path: str) -> tuple:
61             """Return (fps, frame_width) for a video, with sensible fallbacks."""
62             cap = cv2.VideoCapture(video_path)
63             fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
64             width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
65             cap.release()
66             return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
67
68         @staticmethod
69         def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
70             """Provider-reported exchange count, else a duration-based estimate.
71
72             One canonical implementation ? the twins had drifted (wasb relied on
73             ``int(None)`` raising into the fallback; same result, by accident).
74             """
75             shot_count = segment.get("shuttle_exchange_count")
76             if shot_count is None:
77                 return max(3, int(duration / 0.8))
78             try:
79                 return int(shot_count)
80             except (ValueError, TypeError):
81                 return max(3, int(duration / 0.8))
82
83         # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
84         -----
85         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
86             frame_width: float, video_path: str,
87             idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
88             """Stats dict persisted into ``candidate_segments.stats`` for one window. The
89             BASE
90             returns exactly the 4 motion keys, so the trajectory twins persist byte-

```

```

identical
87         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
88         features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
89         return {
90             "peak_velocity": cw["peak_velocity"],
91             "mean_velocity": cw["mean_velocity"],
92             "active_frame_count": cw["active_frame_count"],
93             "track_id_count": cw["track_id_count"],
94         }
95
96     def _select_for_handoff(self, windows: List[Dict[str, Any]],
97                             window_stats: List[Dict[str, Any]],
98                             idx_cfg: Dict[str, Any]) -> List[int]:
99         """Indices of the candidate windows that proceed to the AI handoff (and thus may
100         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102         Persisted candidates are NOT affected ? they remain the full superset for
offline
103         re-scoring."""
104         return list(range(len(windows)))
105
106     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
107                       player_pool: Optional[List[str]] = None,
108                       max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109         # override-ignore: the ABC declares the Result contract (Success|Failure)
110         # but every live segmenter still returns a bare list ? the documented
111         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112         label = self.display_label
113         # --- Sport profile + AI provider wiring (identical across segmenters) ---
114         sport_name = self.config.get("sport", "badminton")
115         try:
116             sport_profile = sport_registry.get(sport_name)
117         except KeyError as e:
118             logger.error(str(e))
119             return []
120
121         idx_cfg = self.config.get("indexing", {})
122         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
123         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
124
125         api_key_env_var = f"{provider_name.upper()}_API_KEY"
126         api_key = os.environ.get(api_key_env_var)
127         if not api_key:
128             logger.error(
129                 f"{api_key_env_var} environment variable is not set! "
130                 f"To run the {provider_name} handoff, set your API key. "
131                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\"")
132             )
133             return []
134         try:
135             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
136         except KeyError as e:
137             logger.error(str(e))
138             return []
139
140         video_duration = get_video_duration(video_path)
141         if video_duration <= 0:
142             logger.error("Invalid video duration.")
143             return []
144         fps, frame_width = self._probe_fps_width(video_path)
145

```

```

146         # --- Runner config + windowing thresholds ---
147         runner, runner_params = self._build_runner(idx_cfg)
148
149         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
150         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
151         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
152         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
153         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154         log_candidates = idx_cfg.get("log_yolo_candidates", True)
155         store_candidates = idx_cfg.get("store_yolo_candidates", True)
156
157         detection_params = {
158             "detector": self.name,
159             "velocity_thresh": velocity_thresh,
160             "merge_gap": merge_gap,
161             "min_action_window_duration": min_window_duration,
162             **runner_params,
163         }
164
165         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
166         ok, msg = runner.healthcheck()
167         if not ok:
168             logger.error("%s WSL environment not ready: %s", label, msg)
169             return []
170         logger.info("%s WSL env OK: %s", label, msg)
171
172         # --- Phase 2: trajectory -> candidate rally windows ---
173         # Trajectory detectors process the whole video in one pass (they carry
174         # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
175         t_start = time.time()
176         out_dir = os.path.join("output", self.family)
177         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
178         if not csv_path:
179             logger.error("%s produced no trajectory; aborting.", label)
180             return []
181
182         points = runner.parse_trajectory_csv(csv_path)
183         logger.info("Parsed %d trajectory points (%d visible).",
184                     len(points), sum(1 for p in points if p.visible))
185
186         all_candidate_windows = runner.trajectory_to_action_windows(
187             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
188             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
189             min_window_duration=min_window_duration, chunk_index=0,
190         )
191         logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
192                     label, time.time() - t_start, len(all_candidate_windows))
193
194         if log_candidates:
195             for cw in all_candidate_windows:
196                 logger.info(f"[{label}] rally
197 {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
198                             f"({cw['end'] - cw['start']:.1f}s) |
199 frames={cw['active_frame_count']} "
200                             f"peak_v={cw['peak_velocity']:.3f}
201 mean_v={cw['mean_velocity']:.3f}")
202
203         # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
204         fusion
205         # segmenter adds net_crossings + person-cue features. Computed once, reused for
206         both
207         # persistence and the handoff gate below.
208         window_stats = [
209             self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
210 idx_cfg)

```

```

205         for cw in all_candidate_windows
206     ]
207
208     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
209     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
210     if store_candidates:
211         for i, cw in enumerate(all_candidate_windows):
212             candidate_ids[i] = self.db.add_candidate_segment(
213                 video_id, detector=self.name,
214                 start_time=cw["start"], end_time=cw["end"],
215                 chunk_index=cw["chunk_index"], confidence=min(1.0,
216                 cw["peak_velocity"]),
217                 stats=window_stats[i], detection_params=detection_params,
218             )
219     if not all_candidate_windows:
220         return []
221
222     # --- Phase 3: AI provider handoff over padded candidate clips ---
223     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
224     # candidates above stay the full superset ? only the served play_segments are
225     # gated.
226     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
227     idx_cfg)
228     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
229     os.makedirs(handoff_dir, exist_ok=True)
230     logger.info("Phase 3: %s validation on %d candidate windows...",
231     provider_name, len(handoff_indices))
232
233     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
234         w_start = max(0, window["start"] - handoff_padding)
235         w_end = min(video_duration, window["end"] + handoff_padding)
236         w_dur = w_end - w_start
237         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
238         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
239         reencode=True):
240             return []
241         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
242         segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
243         label=f"Handoff {wi+1}")
244
245         valid = []
246         for seg in segments:
247             try:
248                 seg["start_time"] = float(seg["start_time"]) + w_start
249                 seg["end_time"] = float(seg["end_time"]) + w_start
250                 seg["_candidate_id"] = candidate_ids[wi]
251                 valid.append(seg)
252             except (ValueError, TypeError, KeyError) as e:
253                 logger.warning("Dropping segment with unparseable timestamps: %s ?
254                 %s", seg, e)
255
256         try:
257             os.remove(w_path)
258         except OSError:
259             pass
260         return valid
261
262     ai_segments: List[dict] = []
263     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
264     executor:
265         futures = [executor.submit(process_candidate_window, wi,
266         all_candidate_windows[wi])
267         for wi in handoff_indices]
268     for future in futures:
269         ai_segments.extend(future.result())

```

```

262         try:
263             os.rmdir(handoff_dir)
264         except OSError:
265             pass
266         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
267
268         # --- Phase 4: DB population + candidate linking ---
269         segments_data = []
270         for segment in ai_segments:
271             try:
272                 start = float(segment["start_time"])
273                 end = float(segment["end_time"])
274                 dur = end - start
275                 metadata = {
276                     "shot_count": self._shot_count_from(segment, dur),
277                     "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
278                     "detector": f"{self.family}-hybrid-{model_name}",
279                 }
280                 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
281                 if segment_id:
282                     candidate_id = segment.get("_candidate_id")
283                     if candidate_id is not None:
284                         self.db.link_candidate_to_segment(candidate_id, segment_id)
285                     segments_data.append({
286                         "id": segment_id, "start_time": start, "end_time": end,
287                         "duration": dur, "metadata": metadata,
288                     })
289             except Exception as e:
290                 logger.warning("Skipping invalid segment struct: %s", e)
291
292         return segments_data
293

```

7. assistant (2026-06-14T06:32:26.285Z)

Now let me verify the registration of the default segmenter and how the scorer/eval harness consumes these features. Let me check what `yolo\_hybrid` is the registered default and search for how `net\_crossings`/person features feed into any learned scorer.

8. user (2026-06-14T06:32:26.878Z)

```

config.json:26:         "default_segmenter": "gemini",
README.md:74:This outputs a validation summary and indexes all rallies directly into the local
SQLite database, using whichever segmenter is set as `default_segmenter` in `config.json`. **The
shipped `config.json` sets `gemini`** (needs `GEMINI_API_KEY`; without it the run silently
produces 0 segments and the observability report flags `silent_provider_noop`). Set it to
`yolo_hybrid`/`wasb_hybrid`/`motion` for the local paths.
README.md:142:         "default_segmenter": "gemini",
README.md:180:*   `indexing.default_segmenter` ? `yolo_hybrid` | `gemini` | `motion`.
README.md:242:   *Or set `"default_segmenter": "yolo_hybrid"` (or `"gemini"`/`"motion"`) under
`"indexing"` in `config.json`.
README.md:246:[Omitted long matching line]
README.md:264:         "default_segmenter": "gemini",
run_process_and_stitch.py:74:         seg_name = cfg.indexing.default_segmenter
backend\config\models.py:112:     default_segmenter: str = "yolo_hybrid"
backend\config\models.py:144:     @field_validator("default_segmenter")
backend\reporting\harvest.py:216:     config_default = (config.get("indexing") or
{}).get("default_segmenter")
backend\main.py:148:     default_seg = getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
backend\main.py:712:     seg_name = args.segmenter or getattr(getattr(global_config,
"indexing", None), "default_segmenter", "motion")

```

```

backend\reporting\spec.yaml:133:      The segmenter that ran differs from config.json's
default_segmenter, and no
tests\test_config.py:23:      assert cfg.indexing.default_segmenter == "yolo_hybrid"
tests\test_config.py:78:      "default_segmenter": "gemini",
tests\test_config.py:89:      assert cfg.indexing.default_segmenter == "gemini"
tests\test_config.py:102:      "indexing": {"default_segmenter": "yolo_hybrid"},
tests\test_config.py:110:      assert cfg.indexing.default_segmenter == "yolo_hybrid"
tests\test_config.py:138:      "indexing": {"default_segmenter": "gemini", "wasb":
{"velocity_threshold": 0.02}},
tests\test_config.py:154:      assert cfg.indexing.default_segmenter == "yolo_hybrid"
tests\test_config.py:198:      assert cfg.get("indexing", {}).get("default_segmenter") ==
"yolo_hybrid"
docs\MULTI_SIGNAL_FUSION_PLAN.md:28:| `yolo_hybrid` (the current `default_segmenter`) | person
velocity (torchvision + ByteTrack) | `backend/pipeline/segmenters/yolo_hybrid.py` |
docs\MULTI_SIGNAL_FUSION_PLAN.md:271: **default-OFF** ? `default_segmenter` unchanged),
subclassing the trajectory engine via two identity
docs\EXPERIMENT_HARNESS.md:106:- `backend/config/models.py::IndexingConfig` ?
`default_segmenter` (:65, default `yolo_hybrid`), `detector_model` (:72),
`detector_score_threshold` (:73), `yolo_velocity_threshold` (:74), `yolo_frame_skip` (:75),
`min_segment_duration` (:68), `dead_time_merge_gap` (:69), `motion_threshold` (:67). **No
`min_crossings` knob yet** (see fusion P2).
docs\EXPERIMENT_HARNESS.md:123:JSON-serializable; selected via the existing `segmenter_registry`
+ `IndexingConfig.default_segmenter`.
docs\EXPERIMENT_HARNESS.md:155:pinned ? **rollback = flip `default_segmenter`/variant config,
never a code revert.**
docs\EXPERIMENT_HARNESS.md:185:3. **Per-cue enable flags** in `IndexingConfig` (default
`False`); `default_segmenter` already exists.
docs\CODE_MAP.md:31: -> seg_name = req.segmenter or global_config.indexing.default_segmenter
(fallback 'motion')
docs\CODE_MAP.md:89: - ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default_segmenter fallback `motion`.
docs\CODE_MAP.md:99:[Omitted long matching line]
docs\CODE_MAP.md:105:- `@config.json` ? on-disk config; mirrors `AppConfig`. ?
**`indexing.default_segmenter='gemini'`** diverges from model default `yolo_hybrid` ? file
wins at runtime.
docs\CODE_MAP.md:186:[Omitted long matching line]
docs\CODE_MAP.md:250:- **Segmenter default:** single source of truth = model
`IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships
`gemini`). run*.py read the model value (no literals); `main.py` keeps the defensive `motion`
getattr fallback noted above.
docs\CODE_MAP.md:276:| Change a config default / add a config field |
`@backend/config/models.py` (single source of truth) -> regenerate via
`setup.py`/`save_default_config`; ? config.json may override (default_segmenter) |
docs\TRACKNET_WSL_SETUP.md:71:Then select it as the segmenter (e.g. set
`indexing.default_segmenter` to
docs\QUALITY_ITERATIONS.md:139:**Shipping shape.** A NEW `fusion` segmenter, **default OFF**
(`default_segmenter` unchanged); Gate A
tests\test_reporting_harvest.py:46:      config={"ai_provider": "gemini", "ai_model": "gemini-
flash", "indexing": {"default_segmenter": "wasb_hybrid"}},
tests\test_reporting_harvest.py:122:      config={"ai_provider": "gemini", "indexing":
{"default_segmenter": "gemini"}},
tests\test_reporting_harvest.py:137:      config={"indexing": {"default_segmenter":
"motion"}})
tests\test_reporting_harvest.py:146:      cfg = {"ai_provider": "gemini", "indexing":
{"default_segmenter": "wasb_hybrid"}}
tests\test_reporting_harvest.py:189:      config={"ai_provider": "gemini", "indexing":
{"default_segmenter": "wasb_hybrid"}},

```

9. user (2026-06-14T06:32:27.702Z)

Found 27 files

```

docs\QUALITY_ITERATIONS.md
tests\test_variant_tuning.py
tests\test_experiment_honest.py

```

```
docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md
docs\EXPERIMENT_HARNESS.md
backend\eval\experiment.py
tests\test_fusion_compare.py
backend\eval\fusion_compare.py
backend\eval\fusion_audio.py
backend\eval\fusion_golden.py
backend\pipeline\detectors\person_detector.py
docs\ANALYZERS_AND_RECONCILERS.md
tests\test_audio_features.py
backend\pipeline\detectors\audio_features.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_rally_gate.py
tests\test_fusion_hybrid.py
tests\test_fusion_features.py
docs\NEXT_STEPS.md
docs\MULTI_SIGNAL_FUSION_PLAN.md
docs\CODE_MAP.md
backend\pipeline\detectors\fusion_features.py
backend\pipeline\detectors\rally_gate.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
tests\test_experiment.py
backend\eval\distill_local.py
```

10. assistant (2026-06-14T06:32:32.608Z)

The default segmenter in the model is `yolo\_hybrid` (not fusion), and config.json ships `gemini` ? both are NOT fusion. Now let me verify the "no_special_casing" and "retained" criteria. The key question: does `net\_crossings`/person enter the DECISION as a learned-scorer feature column, or as a hard-coded if/else? Let me look at how the eval/scorer consumes these features.

11. user (2026-06-14T06:32:33.503Z)

```
1      """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
set.
2
3      The fusion-P3 measurement. Loads the per-video feature JSONs written by
`fusion_golden.py`
4      and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
out) ?
5      no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
harness's
6      designed seam): the person features are extra columns the learned LogisticRegression
weights,
7      so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
8
9      Run (no GPU):
10     python -m backend.eval.fusion_compare --features-dir output --record
11     """
12     from __future__ import annotations
13
14     import argparse
15     import glob
16     import json
17     import os
18     from typing import Any, Dict, List
19
20     from backend.eval import experiment
21     from backend.eval.eval_stats import paired_delta_ci
22     from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25     def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26         """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
```



```

test
27         (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28         by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos.""
29         by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
30         by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
31         vids = [p["video"] for p in res["variant_a"]["per_video"]]
32         return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
33
34
35     def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
36         """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
37         feature columns + golden gts). The harness derives labels from gts itself."""
38         videos: List[experiment.VideoCandidates] = []
39         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
40             with open(path, encoding="utf-8") as f:
41                 d = json.load(f)
42                 cands = d["candidates"]
43                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
44                 features: Dict[str, List[float]] = {}
45                 for fn in SHUTTLE_FEATURE_NAMES:
46                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
47                 for fn in PERSON_FEATURE_NAMES:
48                     features[fn] = [float(c["person"][fn]) for c in cands]
49                 gts = [(float(a), float(b)) for a, b in d["gts"]]
50                 videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
51                                                         features=features, gts=gts))
52         return videos
53
54
55     def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
56                threshold: float = 0.5) -> Dict[str, Any]:
57         shuttle_only = experiment.Variant(name="shuttle_only",
58         feature_names=list(SHUTTLE_FEATURE_NAMES),
59         threshold=threshold)
60         fusion = experiment.Variant(name="shuttle_person_fusion",
61         feature_names=list(SHUTTLE_FEATURE_NAMES) +
62         list(PERSON_FEATURE_NAMES),
63         threshold=threshold)
64         return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)
65
66     def format_result(res: Dict[str, Any], iou: float) -> str:
67         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
68         ds = delta_stats(res)
69         st = ds["sign_test"]
70         lines = [
71             f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
===",
72             f" shuttle-only : F1={a['f1']:.3f} P={a['precision']:.3f} R={a['recall']:.3f}
IoU={a['mean_iou']:.3f}",
73             f" +person : F1={b['f1']:.3f} P={b['precision']:.3f} R={b['recall']:.3f}
IoU={b['mean_iou']:.3f}",
74             f" ?F1 = {d['f1']:+.3f} ({res['delta_pct']['f1']:+.1f}%)
?P={d['precision']:+.3f} ?R={d['recall']:+.3f}",
75             f" paired ?F1 {ds['mean_delta']:+.3f} [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}] "
76             f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f}) "
77             f"| CI clears 0: {ds['ci_excludes_zero']}",
78             " per-video ?F1:",
79         ]
80         for pv in res["per_video_delta"]:
81             lines.append(f" {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
82         return "\n".join(lines)

```

```

82
83
84     def main() -> None:
85         ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
shuttle+person (golden set).")
86         ap.add_argument("--features-dir", default="output")
87         ap.add_argument("--iou", type=float, default=0.5)
88         ap.add_argument("--record", action="store_true", help="append the result to the
experiment leaderboard")
89         args = ap.parse_args()
90
91         videos = load_videos(args.features_dir)
92         if len(videos) < 2:
93             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
{len(videos)})")
94         res = compare(videos, iou=args.iou)
95         print(format_result(res, args.iou))
96         if args.record:
97             row = experiment.record_experiment(res)
98             print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']}")
99
100
101     if __name__ == "__main__":
102         main()
103

```

12. assistant (2026-06-14T06:32:37.809Z)

This confirms the features enter the decision as feature COLUMNS that a learned LogisticRegression weights. Let me verify the experiment harness `Variant`/`compare` actually trains a learned scorer over those feature columns (not a hard-coded threshold), and confirm `net\_crossings` is a column in the shuttle feature set.

13. user (2026-06-14T06:32:38.774Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. **No new scoring math lives
7     here**; everything defers to:
8
9     - `backend.eval.metrics`           ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`           ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`         ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine`      ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`         ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`        ? `pct_improvement`
15
16    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
17    (per-frame signals ? run once on the GPU/WSL box, persisted into
18    `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
19    signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20    given persisted candidate features it produces `List[Interval]` deterministically,
21    with no GPU and zero production risk. So most experiments are pure offline
22    re-scoring of stored data and never touch the served pipeline.
23
24    Three modes (`EXPERIMENT_HARNESS.md` §5):
25    - `compare()`           ? labeled A/B vs golden labels: per-video + LOVO-honest
26      aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27      held-out loop, but variant-parameterised.
28    - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29      variants agree / diverge (A-only, B-only, agreed). The divergences are what a

```

```

30     human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32     (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**
33
34 Pure + offline + unit-tested. The only DB-touching helper is
35 `load_video_candidates`, which reads persisted candidates via
36 `Database.query_candidate_segments`.
37 """
38 from __future__ import annotations
39
40 import json
41 import logging
42 import os
43 from dataclasses import dataclass, field
44 from datetime import datetime, timezone
45 from hashlib import sha1
46 from typing import Any, Callable, Dict, List, Optional, Sequence
47
48 from backend.eval.calibration import pct_improvement
49 from backend.eval.classifier import LogisticRegression
50 from backend.eval.gemini_refine import merge_overlaps
51 from backend.eval.metrics import Interval, match_intervals, score
52 from backend.eval.regression import gt_hash
53 from backend.eval.training import label_candidates
54 from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
55 from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count
ratio
56 from backend.eval.score_calibration import fit_isotonic, fit_platt  # C2: calibrate cue
scores
57
58 logger = logging.getLogger(__name__)
59
60 # Where a comparison run appends one reproducible record. Tracked location (NOT under
61 # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
62 # regression.DEFAULT_BASELINE_PATH.
63 DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
64 _METRICS = ("precision", "recall", "f1", "mean_iou")
65
66
67 class ExperimentError(ValueError):
68     """A variant cannot be evaluated as configured (e.g. a learned variant asked to
69     score an unlabeled video with no pinned model, or a gate referencing an absent
70     feature)."""
71
72
73 # ----- Variant
74
75
76 @dataclass
77 class Variant:
78     """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80     A variant turns one video's persisted candidate windows + features into rally
81     intervals. Every knob defaults to the **control** behaviour (no gate, no learned
82     filter), so a variant that merely *carries* a new, disabled cue is byte-identical
83     to control ? the core no-regression guarantee.
84
85     Fields:
86     - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87       for the leaderboard and for selecting the served path (the existing
88       `segmenter_registry` + `IndexingConfig` do the actual selection in production).
89       New cues are recorded here default-OFF.
90     - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
91       ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
92       `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-

```

```

shuttle
93         windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94     - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
95       whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
96       cue persists that feature).
97     - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
98       `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
99       required for `compare_unlabeled` on a learned variant.
100    - ``feature_names`` ? the persisted features the learned filter consumes. When set
101      WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
102      (honest) ? the recipe, not a pinned artifact.
103    - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104      overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
105    ""
106
107    name: str
108    segmenter: str = "wasb_hybrid"
109    config_overrides: Dict[str, Any] = field(default_factory=dict)
110    cue_flags: Dict[str, bool] = field(default_factory=dict)
111    min_crossings: int = 0
112    min_players: int = 0
113    classifier_model: Optional[str] = None
114    feature_names: Optional[List[str]] = None
115    threshold: float = 0.5
116    merge_gap: float = 1.0
117    # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118    # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119    # failure where a fixed 0.5 zeroed out a small-candidate video.
120    tune_threshold: bool = False          # pick the F1-best threshold on the train fold
121    calibrate: Optional[str] = None        # None | "platt" | "isotonic" ? calibrate
proba first
122
123    def is_learned(self) -> bool:
124        """True if this variant applies a learned classifier (pinned model or
recipe)."""
125        return self.classifier_model is not None or bool(self.feature_names)
126
127    def to_dict(self) -> dict:
128        return {
129            "name": self.name,
130            "segmenter": self.segmenter,
131            "config_overrides": self.config_overrides,
132            "cue_flags": self.cue_flags,
133            "min_crossings": self.min_crossings,
134            "min_players": self.min_players,
135            "classifier_model": self.classifier_model,
136            "feature_names": self.feature_names,
137            "threshold": self.threshold,
138            "merge_gap": self.merge_gap,
139            "tune_threshold": self.tune_threshold,
140            "calibrate": self.calibrate,
141        }
142
143    @classmethod
144    def from_dict(cls, d: dict) -> "Variant":
145        known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146        return cls(**{k: v for k, v in d.items() if k in known})
147
148    def spec_hash(self) -> str:
149        """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150        and makes a result reproducible. The pinned **control** variant always hashes
the
151        same, so a regression is always traceable to a recipe change, never to drift."""

```

```

152         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
153         return sha1(blob.encode()).hexdigest()[:12]
154
155
156 #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157 #: filter. Always the comparison reference; "rollback" = make this the served variant.
158 CONTROL = Variant(name="control")
159
160
161 # ----- persisted candidates
162
163
164 @dataclass
165 class VideoCandidates:
166     """One video's persisted detection, ready for offline re-scoring.
167
168     ``intervals`` are the candidate windows; ``features`` is column-major
169     (``feature_name`` -> per-candidate value, each list aligned to ``intervals``);
170     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171     with ``load_video_candidates``, or directly in tests.
172     """
173
174     name: str
175     intervals: List[Interval]
176     features: Dict[str, List[float]] = field(default_factory=dict)
177     gts: Optional[List[Interval]] = None
178
179
180 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181     """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182     `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
183     col = vc.features.get(name)
184     n = len(vc.intervals)
185     if col is None:
186         logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                        name, vc.name)
188         return [0.0] * n
189     if len(col) != n:
190         raise ExperimentError(
191             f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192     return [float(x) for x in col]
193
194
195 def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
196     cols = [_feature_column(vc, name) for name in feature_names]
197     return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
198
199
200 def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201     """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
202     players)."""
203     n = len(vc.intervals)
204     mask = [True] * n
205     if variant.min_crossings > 0:
206         col = _feature_column(vc, "net_crossings")
207         mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
208     if variant.min_players > 0:
209         col = _feature_column(vc, "players_in_court")
210         mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
211     return mask
212
213
214 def predict(variant: Variant, vc: VideoCandidates,
215            model: Optional[LogisticRegression] = None,

```

```

215         threshold: Optional[float] = None,
216         calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
217     """Variant prediction: gate by persisted features, optionally filter by a learned
218     model, then merge. Pure and deterministic.
219
220     ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
221     fold; if omitted and the variant pins ``classifier_model``, that frozen model is
222     loaded. A learned variant with neither a supplied nor a pinned model raises ? there
223     is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
224     fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
225     them.
226     """
227     mask = _gate_mask(variant, vc)
228     if variant.feature_names:
229         if model is None:
230             if variant.classifier_model is None:
231                 raise ExperimentError(
232                     f"variant {variant.name!r} is learned but has no model to predict "
233                     f"with (pass a fitted model or set classifier_model)")
234             model = LogisticRegression.load(variant.classifier_model)
235             proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
236 variant.feature_names))]
237             if calibrator is not None:
238                 proba = [calibrator(p) for p in proba]
239                 thr = variant.threshold if threshold is None else threshold
240                 mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
241             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
242             return merge_overlaps(kept, gap_tol=variant.merge_gap)
243
244     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
245                             train_videos: Sequence[VideoCandidates], iou: float
246                             ) -> "tuple[Optional[Callable[[float], float]],
247 Optional[float]]":
248         """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
249         (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
250         (use the variant default). Honest: never looks at the held-out video.
251         """
252         calibrator: Optional[Callable[[float], float]] = None
253         if variant.calibrate:
254             raw: List[float] = []
255             y: List[int] = []
256             for vc in train_videos:
257                 raw.extend(float(p) for p in
258 model.predict_proba(_feature_matrix(vc, variant.feature_names or
259 [])))
260                 y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
261             if variant.calibrate == "platt":
262                 calibrator = fit_platt(raw, y)
263             elif variant.calibrate == "isotonic":
264                 calibrator = fit_isotonic(raw, y)
265             else:
266                 raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
267 'platt'/'isotonic')")
268             threshold: Optional[float] = None
269             if variant.tune_threshold:
270                 best_t, best_f1 = variant.threshold, -1.0
271                 for k in range(1, 20):
272                     # grid 0.05..0.95 on the train
273                     t = round(0.05 * k, 2)
274                     f1s = [score(predict(variant, vc, model=model, threshold=t,
275 calibrator=calibrator),
276 vc.gts or [], iou).f1 for vc in train_videos]
277                     mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
278                     if mean_f1 > best_f1:

```

```

273         best_f1, best_t = mean_f1, t
274         threshold = best_t
275     return calibrator, threshold
276
277
278     # ----- variant scoring
279
280
281     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
282         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
283         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
284         X: List[List[float]] = []
285         y: List[int] = []
286         for vc in videos:
287             if vc.gts is None:
288                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
289                 X.extend(_feature_matrix(vc, variant.feature_names or []))
290                 y.extend(label_candidates(vc.intervals, vc.gts, iou))
291             if not X:
292                 raise ExperimentError("cannot train a learned variant on zero candidates")
293         return LogisticRegression().fit(X, y, variant.feature_names)
294
295
296     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
297                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
298         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
299         mean aggregate.
300
301         Prediction policy:
302         - **static variant** (no learned filter): predict each video directly ? no
training,
303         so no leakage; per-video scoring is already honest.
304         - **learned, pinned model** (`classifier_model` set): score every video with the
305         SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
306         report "how the shipped artifact does here", not for the promotion decision.
307         - **learned recipe** (`feature_names`, no pinned model) + `honest=True`: LOVO
?
308         train on the OTHER videos, predict the held-out one. The number to trust.
309         - learned recipe + `honest=False`: train on all, predict each (overfit; sanity
only).
310         """
311         for vc in videos:
312             if vc.gts is None:
313                 raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
314
315         per_video: List[Dict[str, Any]] = []
316         predictions: Dict[str, List[Interval]] = {}
317
318         recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
319         pinned_model = (LogisticRegression.load(variant.classifier_model)
320                         if variant.classifier_model is not None else None)
321         all_model = None
322         if recipe_lovo and not honest:
323             all_model = _train(variant, videos, iou)
324
325         for i, vc in enumerate(videos):
326             cal: Optional[Callable[[float], float]] = None
327             thr: Optional[float] = None
328             if recipe_lovo and honest:
329                 others = [v for j, v in enumerate(videos) if j != i]
330                 if not others:

```

```

331         raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332         model: Optional[LogisticRegression] = _train(variant, others, iou)
333         if variant.tune_threshold or variant.calibrate: # operating point from
the train fold
334             assert model is not None # _train never returns
None
335             cal, thr = _calibrate_and_tune(variant, model, others, iou)
336         elif recipe_lovo: # honest=False ? one model over all
videos
337             model = all_model
338         else: # static variant or pinned model
339             model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
for m in _METRICS}
347         for m in _METRICS}
348         return {
349             "variant": variant.name,
350             "spec_hash": variant.spec_hash(),
351             "is_learned": variant.is_learned(),
352             "honest_lovo": recipe_lovo and honest,
353             "per_video": per_video,
354             "aggregate": aggregate,
355             "predictions": predictions,
356         }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
361         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}
367         overlaps = _segm.DEFAULT_OVERLAPS
368         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
369         ratios: List[float] = []
370         for name, preds in eval_v.get("predictions", {}).items():
371             gts = gts_by_name.get(name, [])
372             got = _segm.f1_at_overlaps(preds, gts, overlaps)
373             for ov in overlaps:
374                 f1k[ov].append(got[ov])
375             ratios.append(_segm.segment_count_ratio(preds, gts))
376
377         def _mean(xs: List[float]) -> float:
378             return sum(xs) / len(xs) if xs else 0.0
379         out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380         out["segment_count_ratio"] = _mean(ratios)
381         return out
382
383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],

```



```

385         eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).
387
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391         """
392         a_f1 = [p["f1"] for p in eval_a["per_video"]]
393         b_f1 = [p["f1"] for p in eval_b["per_video"]]
394         return {
395             "variant_a_ci": _ci_block(eval_a),
396             "variant_b_ci": _ci_block(eval_b),
397             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
399                             "variant_b": _segmentation_block(videos, eval_b)},
400         }
401
402
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404                 iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (EXPERIMENT_HARNESS.md §5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416         bare LOVO mean at n?6 is not trustworthy on its own.
417         """
418         if len(videos) < 1:
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427         per_video_delta = [
428             {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429             for p in eval_b["per_video"]
430         ]
431
432         # One fingerprint over the whole label set ? detects "the deltas were measured
against
433         # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434         all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
```

```

435
436     result: Dict[str, Any] = {
437         "iou": iou,
438         "label_set_hash": gt_hash(all_gts),
439         "num_videos": len(videos),
440         "variant_a": eval_a,
441         "variant_b": eval_b,
442         "delta": delta,
443         "delta_pct": delta_pct,
444         "per_video_delta": per_video_delta,
445     }
446     if honest_stats:
447         result["honest"] = honest_summary(videos, eval_a, eval_b)
448     return result
449
450
451 # ----- SxS on unlabeled video
452
453
454 def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455                       agree_iou: float = 0.5) -> Dict[str, Any]:
456     """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458     With no labels there is no lift to measure ? instead this surfaces where the two
459     variants **agree vs diverge**, which is exactly what a human reviews (and what two
460     highlight reels would show side by side):
461
462     - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463     - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464     - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPS:
465       the human / a later golden label adjudicates).
466
467     Both variants must be predictable without labels: a learned variant therefore needs
468     a
469     pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
470     ``metrics.match_intervals`` ? no new matching logic.
471
472     """
473     preds_a = predict(variant_a, video)
474     preds_b = predict(variant_b, video)
475     mr = match_intervals(preds_a, preds_b, agree_iou)
476
477     agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
478               for m in mr.matches]
479     agree_ious = [m.iou for m in mr.matches]
480     a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
481     b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
482
483     def _dur(ivs: List[Interval]) -> float:
484         return sum(e - s for s, e in ivs)
485
486     return {
487         "video": video.name,
488         "agree_iou": agree_iou,
489         "variant_a": variant_a.name,
490         "variant_b": variant_b.name,
491         "num_a": len(preds_a),
492         "num_b": len(preds_b),
493         "num_agreed": len(agreed),
494         "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
495         "duration_a": _dur(preds_a),
496         "duration_b": _dur(preds_b),
497         "agreed": agreed,
498         "a_only": a_only,
499         "b_only": b_only,
500     }

```

```

499
500
501 # ----- leaderboard / DB
502
503
504 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                      timestamp: Optional[str] = None) -> Dict[str, Any]:
506     """Append a compact, reproducible record of a `compare()` result to the JSON
507     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508     `regression_baseline.json`; deliberately the size of a baseline file, not a service
509     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510     """
511     row: Dict[str, Any] = {
512         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
513         "iou": result.get("iou"),
514         "label_set_hash": result.get("label_set_hash"),
515         "num_videos": result.get("num_videos"),
516         "variant_a": {"name": result["variant_a"]["variant"],
517                      "spec_hash": result["variant_a"]["spec_hash"],
518                      "aggregate": result["variant_a"]["aggregate"]},
519         "variant_b": {"name": result["variant_b"]["variant"],
520                      "spec_hash": result["variant_b"]["spec_hash"],
521                      "aggregate": result["variant_b"]["aggregate"]},
522         "delta": result.get("delta"),
523     }
524     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
525     # the
526     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
527     honest = result.get("honest") or {}
528     if honest.get("paired_delta_f1"):
529         row["honest_delta_f1"] = honest["paired_delta_f1"]
530     entries: List[Dict[str, Any]] = []
531     if os.path.isfile(path):
532         with open(path, encoding="utf-8") as f:
533             entries = json.load(f)
534     entries.append(row)
535     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
536     with open(path, "w", encoding="utf-8") as f:
537         json.dump(entries, f, indent=2)
538     logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
539               row["variant_a"]["name"], row["variant_b"]["name"],
540               (result.get("delta") or {}).get("f1", 0.0), path)
541     return row
542
543 def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                          gts: Optional[List[Interval]] = None,
545                          feature_names: Sequence[str] = ()) -> VideoCandidates:
546     """Bridge persisted detection ? `VideoCandidates` via
547     `Database.query_candidate_segments`.
548     ``feature_names`` selects which persisted features to lift out of each row's
549     ``stats``
550     JSON (``duration`` is derived from the window bounds). This is the only DB-
551     touching
552     function; the comparison itself is pure and offline. Exercised owner-side on a real
553     DB,
554     but works against any `Database` (including an in-memory one in tests).
555     """
556     rows = db.query_candidate_segments(video_id, detector=detector)
557     intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
558     rows]
559     features: Dict[str, List[float]] = {}
560     for name in feature_names:
561         if name == "duration":

```

```

558             features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559         else:
560             features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561     return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562

```

14. assistant (2026-06-14T06:32:45.392Z)

This is comprehensive. Now I understand the architecture fully. Let me verify the "retained" criterion by checking the n=6 ?F1 result history and confirming nothing was deleted/disabled. Let me look at the experiment leaderboard and the fusion_golden feature names, plus check git log for the n=6 result commit.

15. user (2026-06-14T06:32:47.375Z)

```

30-from backend.eval.training import label_candidates
31-
32-# The 5 fps/resolution-invariant shuttle features distill_local already computes.
33:SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
34:# The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
35:PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
36-
37-
38-def _derive_video_path(spec: Dict[str, Any]) -> str:
39-
40-
41-
42-
43-
44-
45-
46-
47-
48-
49-
50-
51-
52-
53-
54-
55-
56-
57-
58-
59-
60-
61-
62-
63-
64-
65-
66-
67-
68-
69-
70-
71-
72-
73-
74-
75-
76-
77-
78-
79-
80-
81-
82-
83-
84-
85-
86-
87-
88-
89-
90-
91-
92-
93-
94-
95-
96-
97-
98-
99-
100-
101-
102-
103-
104-
105-
106-
107-
108-
109-
110-
111-
112-
113-
114-
115-

```

16. user (2026-06-14T06:32:48.601Z)

d7b1f51 feat(eval): honest ?F1 ? bootstrap CI + paired sign test on the fusion lift (#127)
39e961b feat(eval): fusion P3 measurement ? golden person-feature extractor + LOVO compare (#121)
cbf2236 feat(segmenters): FusionSegmenter ? shuttle + net-crossing + person-cue fusion (P2, default-OFF) (#117)

17. assistant (2026-06-14T06:32:51.805Z)

Now let me check the experiment leaderboard for the actual n=6 ?F1 result and confirm the producer/feature math is still intact (not deleted post-measurement). Let me also confirm `net\_crossings` is recognized in the shuttle feature names or available as a gate column.

18. user (2026-06-14T06:32:53.574Z)

```

---LEADERBOARD---
[
  {
    "timestamp": "2026-06-14T06:23:51.507393+00:00",
    "iou": 0.5,
    "label_set_hash": "50d98ffbc370",
    "num_videos": 6,
    "variant_a": {
      "name": "shuttle_only",
      "spec_hash": "47c867c36f2f",
      "aggregate": {
        "precision": 0.31807603780927995,

```

```

    "recall": 0.3101947096042718,
    "f1": 0.3068611057861412,
    "mean_iou": 0.5791831889148886
  },
  "variant_b": {
    "name": "shuttle_person_fusion",
    "spec_hash": "253f4709b4c4",
    "aggregate": {
      "precision": 0.3617228001510626,
      "recall": 0.2708939086387243,
      "f1": 0.2863398846047612,
      "mean_iou": 0.5690111885714347
    }
  },
  "delta": {
    "precision": 0.04364676234178266,
    "recall": -0.0393008009655475,
    "f1": -0.02052122118138,
    "mean_iou": -0.01017200034345389
  },
  "honest_delta_f1": {
    "mean_delta": -0.02052122118137999,
    "ci_low": -0.16462806145542527,
    "ci_high": 0.16224446866060574,
    "n": 6.0,
    "ci_excludes_zero": false,
    "sign_test": {
      "wins": 2.0,
      "losses": 4.0,
      "ties": 0.0,
      "n_effective": 6.0,
      "p_value": 0.6875
    }
  },
},
{
  "timestamp": "2026-06-14T06:24:56.487711+00:00",
  "iou": 0.5,
  "label_set_hash": "50d98ffbc370",
  "num_videos": 6,
  "variant_a": {
    "name": "shuttle_person",
    "spec_hash": "ee89f1e41604",
    "aggregate": {
      "precision": 0.3617228001510626,
      "recall": 0.2708939086387243,
      "f1": 0.2863398846047612,
      "mean_iou": 0.5690111885714347
    }
  },
  "variant_b": {
    "name": "shuttle_person_audio",
    "spec_hash": "d68c39eba87c",
    "aggregate": {
      "precision": 0.4530798177857001,
      "recall": 0.3408138120839734,
      "f1": 0.3655826552797436,
      "mean_iou": 0.7074802852140344
    }
  },
  "delta": {
    "precision": 0.0913570176346375,
    "recall": 0.06991990344524907,
    "f1": 0.07924277067498242,

```

```

    "mean_iou": 0.13846909664259965
},
"honest_delta_f1": {
    "mean_delta": 0.07924277067498244,

```

19. user (2026-06-14T06:32:54.322Z)

```

38-Interval = Tuple[float, float]
39-
40:FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
41-# Recall-oriented proposal: deliberately permissive so real rallies are among the
42-# candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
--
114-        X = [row for v in others for row in v["X"]]
115-        y = [t for v in others for t in v["y"]]
116-        model = LogisticRegression().fit(X, y, FEATURE_NAMES)
117-        preds = predict_rallies(model, held["X"], held["intervals"], threshold)
118-        sc = score(preds, held["gts"], iou)
--
133-    Xall = [row for v in videos for row in v["X"]]
134-    yall = [t for v in videos for t in v["y"]]
135-    final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
136-    assert final.w is not None # set by fit()
137-    weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
138-    print(f"\nfinal model standardized weights (sign\size = influence): {weights}")
139-    if model_out:

```

20. assistant (2026-06-14T06:33:00.041Z)

This fully confirms all four criteria. The n=6 measured ?F1 was -0.021 (CI [-0.165, +0.162], excludes-zero: false, sign test 2W/4L p=0.6875) ? non-significant. And yet the code (``fusion_hybrid.py``, the feature producers, the gate columns) is all still present and re-measurable. ``net_crossings`` is even one of the 5 shuttle features the learned LogisticRegression weights.

Let me do a final verification that the fusion segmenter is registered but not the default anywhere, and confirm the person-cue producer (``fusion_features.py``) still exists intact.

21. user (2026-06-14T06:33:04.287Z)

```

=== fusion_features.py exists & PERSON_FEATURE_NAMES ===
backend/eval/fusion_compare.py
backend/eval/fusion_golden.py
backend/pipeline/detectors/fusion_features.py
backend/pipeline/detectors/rally_gate.py
=== grep PERSON_FEATURE_NAMES def ===
194:PERSON_FEATURE_NAMES = [
206:     shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats.""
=== config.json default_segmenter ===
26:     "default_segmenter": "gemini",
=== is fusion ever a default anywhere? ===
backend/pipeline/segmenters/fusion_hybrid.py:56:         return "fusion_hybrid"

```

22. assistant (2026-06-14T06:33:08.484Z)

All four criteria are verified. The only place ``"fusion_hybrid"`` appears as a value is the segmenter's own ``name`` property ? never as a default. `config.json` ships ``gemini``, model default is ``yolo_hybrid``. Both producers (``fusion_features.py``, ``rally_gate.py``) and both eval drivers (``fusion_golden.py``, ``fusion_compare.py``) are tracked and intact.

I have enough evidence to give a precise, skeptical verdict.

23. user (2026-06-14T06:33:41.159Z)

Structured output provided successfully

24. assistant (2026-06-14T06:33:50.072Z)

I've completed the audit of the FusionSegmenter + config defaults component. All four criteria verified TRUE, with concerns noted. The structured output has been returned.

Summary of findings:

- **present**: `FusionSegmenter` exists at `fusion\_hybrid.py:39`, registered at `:154`; producers (`fusion\_features.py`, `rally\_gate.py`) and eval drivers (`fusion\_golden.py`, `fusion\_compare.py`) all git-tracked and intact.
- **default_off**: model default `yolo\_hybrid` (`models.py:112`), shipped config.json `gemini` (`:26`) ? neither is fusion. `cue\_flags` defaults to empty dict ? person OFF; `min\_crossings`/`min\_players` default 0 ? served Gate A/B is a no-op, identical to the substrate.
- **no_special_casing**: in the eval harness the cues enter as feature COLUMNS a `LogisticRegression` weights (`fusion\_compare.py:57-61`, `experiment.py:227-238/281-293`); `net\_crossings` is one of the 5 learned shuttle features (`distill\_local.py:40`).
- **retained**: the n=6 result was non-significant (leaderboard ?F1 = ?0.021, CI excludes-zero false, sign test 2W/4L p=0.6875) yet nothing was deleted/disabled ? the producer + feature math still compute and persist, and the persisted-candidate superset invariant keeps it re-measurable as the corpus grows.

Key caveat surfaced in concerns: the *served* Gate A/B path (`\_select\_for\_handoff`) is a hard if/else threshold branch, but it's default-OFF and the learned-scorer treatment lives in the offline eval harness ? so the no-special-casing guarantee holds for how the cue enters the learned decision.